

1. Introducción y motivación

El objetivo de esta tarea es integrar dos algoritmos de predicción desarrollados en el marco del proyecto PERTE VEC para ofrecer al usuario un tiempo de llegada total más realista y completo. Hasta ahora, el modelo de predicción de tráfico desarrollado por Laura Rodríguez estimaba únicamente el tiempo de conducción desde el origen hasta el área de destino. Sin embargo, en un viaje urbano en vehículo eléctrico, el tiempo de conducción es solo una parte del tiempo total que el usuario necesita para llegar a su destino final. A este tiempo hay que añadir el tiempo que tardará en encontrar y maniobrar en un aparcamiento, y el tiempo que tardará en caminar desde el parking hasta su destino.

Esta integración responde a una necesidad real e identificada: los sistemas de navegación actuales son reactivos, no predictivos, y no tienen en cuenta la dificultad de aparcamiento en el área de destino a la hora de calcular el tiempo de llegada. Esto genera estimaciones optimistas que no reflejan la experiencia real del conductor, especialmente en zonas urbanas densas como Madrid durante las horas punta.

La solución desarrollada combina tres componentes:

- **El modelo LSTM de predicción de tráfico**, que estima la intensidad y la velocidad media en la red viaria y a partir de ellas calcula el tiempo de conducción.
- **El algoritmo NEXTPARK**, que a partir del comportamiento cinemático del vehículo en los últimos minutos predice cuánto tiempo tardará el conductor en encontrar aparcamiento.
- **Un módulo de cálculo de la caminata**, que estima el tiempo que el conductor tardará en ir a pie desde el parking hasta su destino final.

El resultado es un **ETA holístico** que desglosa el tiempo total del trayecto en tres componentes: conducción, maniobra de aparcamiento y caminata.

2. Descripción de los algoritmos de entrada

2.1 Modelo LSTM de predicción de tráfico

El modelo de predicción de tráfico fue desarrollado por Laura Rodríguez como parte del proyecto PERTE VEC. Se trata de una red neuronal de tipo LSTM (Long Short-Term Memory), especialmente adecuada para el modelado de series temporales con dependencias a largo plazo, como los patrones de tráfico urbano.

El modelo toma como entrada un historial reciente de mediciones de un punto concreto de la red viaria de Madrid (intensidad, ocupación, carga y velocidad media) junto con variables cíclicas temporales que capturan la periodicidad del tráfico (hora del día, día de la semana y mes del año, codificados mediante funciones seno y coseno). A partir de estas entradas, el modelo predice dos variables para ese punto: la intensidad de tráfico (vehículos por hora) y la velocidad media (km/h).

La arquitectura seleccionada tras la comparación de cuatro variantes es un modelo complejo con dos capas LSTM (128 y 64 neuronas respectivamente), capas densas de regularización con Dropout y penalización L2, y la transformación cíclica de las variables temporales. Esta arquitectura obtuvo un R^2 de 0,4733 sobre el conjunto de validación, con un MAE de 32,38

vehículos/hora y un RMSE de 64,73 vehículos/hora. Si bien estos valores reflejan un margen de mejora, el modelo es suficientemente robusto para su uso en el cálculo del ETA holístico, especialmente considerando que las proyecciones de mejora con variables externas (meteorología, incidencias DGT) estiman un R^2 cercano al 80%.

En la integración, el modelo LSTM actúa como la primera capa del cálculo: a partir de la velocidad media predicha para el nodo de destino, se estima el tiempo de conducción necesario para llegar al área de destino. En producción, este cálculo se realizará tramo a tramo mediante el algoritmo de Dijkstra sobre el grafo de la red viaria, utilizando las predicciones de intensidad de cada nodo como pesos de las aristas.

2.2 Algoritmo NEXTPARK

NEXTPARK es un algoritmo de predicción de la intención de aparcamiento desarrollado por Next Mobility Solutions. A diferencia del modelo de tráfico, que trabaja sobre datos agregados de la red viaria, NEXTPARK opera sobre el historial de movimiento individual de cada vehículo en los últimos minutos, detectando patrones cinemáticos que indican que el conductor está buscando aparcamiento.

El modelo subyacente es una regresión lineal con transformación logarítmica de la variable respuesta (tiempo hasta aparcar, en segundos), entrenada sobre aproximadamente 41.000 viajes reales. Las variables predictoras del modelo definitivo son cinco, cada una calculada sobre una ventana temporal específica del historial del viaje:

- **Área cubierta** (últimos 20 min): área del convex hull de los puntos GPS recorridos. Un área pequeña y decreciente indica que el vehículo está moviéndose en círculos en una zona reducida, señal característica de la búsqueda de aparcamiento.
- **Variación de velocidad** (últimos 10 min): desviación estándar de la velocidad calculada a partir de los puntos GPS. Una variación alta con velocidad media baja indica frenadas y arrancadas típicas de la búsqueda de plaza.
- **Amplitud del ángulo de giro** (últimos 5 min): diferencia entre el máximo y el mínimo ángulo de orientación del vehículo. Muchos giros en poco tiempo indican maniobras de búsqueda.
- **Distancia recorrida** (últimos 5 min): distancia total recorrida en km. Una distancia baja con muchos giros refuerza la señal de búsqueda de aparcamiento.
- **Velocidad ventana** (últimos 5 min): velocidad media en los últimos 5 minutos. El umbral de 40 km/h es el punto a partir del cual la confianza de la predicción es mayor.

La fórmula del modelo es:

$$\log(\text{time_to_park} + 1) = 6.025 - 0.013 \cdot \text{área} - 0.004 \cdot \text{var_vel} - 0.003 \cdot \text{amplitud} + 0.101 \cdot \text{distancia} - 0.007 \cdot \text{vel_ventana}$$

El resultado es el tiempo estimado en segundos hasta que el conductor encontrará aparcamiento. La alarma de intención de aparcamiento se activa cuando este tiempo es inferior a 10 minutos, la velocidad ventana es menor de 40 km/h y el conductor no está en una zona de aparcamiento frecuente (zona donde habitualmente aparca, detectada mediante un radio de 250 metros sobre sus puntos frecuentes históricos).

3. Diseño de la integración

3.1 La fórmula del ETA holístico

El ETA holístico se define como la suma de tres componentes temporales:

$$\text{ETA}_{\text{total}} = T_{\text{conducción}} + T_{\text{maniobra}} + T_{\text{caminata}}$$

Cada componente tiene una fuente de datos y un método de cálculo diferente:

- **T_conducción** procede directamente del modelo LSTM. A partir de la velocidad media predicha para el nodo de destino y la distancia estimada al área de destino, se calcula el tiempo de conducción en minutos.
- **T_maniobra** es un parámetro fijo de 4 minutos que representa el tiempo medio de rampa de acceso al parking y maniobra de estacionamiento. Este valor fue determinado a partir del análisis del comportamiento de conductores en parkings urbanos y es consistente con los valores utilizados en estudios de movilidad urbana similares.
- **T_caminata** se calcula a partir de la distancia en línea recta entre el parking seleccionado y el destino final del usuario, dividida por la velocidad media de caminata de 4,8 km/h (valor estándar utilizado en planificación urbana y consistente con los estudios de movilidad peatonal).

3.2 Selección del parking óptimo

El motor de integración recibe como entrada una lista de parkings disponibles en el área de destino. Para cada parking se aplica el siguiente proceso de selección:

En primer lugar, se aplica un **filtro de ocupación crítica**: se descartan todos los parkings con una ocupación igual o superior al 95%, ya que la probabilidad de encontrar plaza libre en esas condiciones es prácticamente nula. Este umbral fue seleccionado porque por encima del 95% de ocupación el tiempo de búsqueda de plaza dentro del parking supera en media los 10 minutos adicionales, lo que hace que la estimación del ETA sea poco fiable.

Para los parkings restantes, se calcula el ETA holístico completo y se selecciona el que minimiza el tiempo total. Esta decisión es deliberada: se optimiza el tiempo total de llegada al destino final, no únicamente el tiempo de conducción ni la proximidad del parking. Un parking más cercano al usuario pero más alejado del destino puede resultar en un ETA total mayor que un parking más lejano para el conductor pero más próximo al destino.

3.3 Cálculo de la caminata

El tiempo de caminata se calcula utilizando la librería geopy, que proporciona el cálculo de distancias geodésicas precisas entre dos puntos geográficos expresados en coordenadas (latitud, longitud). Se utiliza la distancia geodésica en lugar de la distancia euclídea porque, aunque las distancias son relativamente cortas (del orden de cientos de metros), la precisión del cálculo es importante para la correcta estimación del tiempo de caminata.

La decisión de usar la distancia en línea recta y no la distancia real de caminata (por las calles) es una simplificación consciente y justificada: en entornos urbanos densos como Madrid, la distancia real por las calles suele ser entre un 20% y un 30% mayor que la distancia en línea recta. Para compensar esta simplificación sin añadir dependencias externas

(como una API de rutas peatonales), se podría aplicar un factor de corrección de 1,25 sobre la distancia calculada. En la versión actual del motor este factor no está aplicado, pero está documentado como una mejora a incorporar en la siguiente iteración.

El punto de inicio de la caminata es la posición del parking seleccionado, y el punto de destino es la posición actual del vehículo (última coordenada del historial de viaje), que se usa como proxy del destino final del usuario. En producción, el destino explícito introducido por el usuario en la app sustituirá a esta aproximación.

4. Indicadores de experiencia de usuario

Además del ETA holístico, el motor genera un conjunto de indicadores visuales diseñados para ofrecer al usuario información útil de forma inmediata y sin necesidad de interpretar números. Todos los indicadores siguen la lógica de semáforo (verde/amarillo/rojo) propuesta por Javier y están optimizados para su visualización en la app móvil y en CarPlay.

4.1 Semáforo de aparcamiento

El semáforo de aparcamiento traduce el tiempo estimado hasta aparcar (resultado de NEXT PARK) en un color:

- **Verde** (< 10 min): aparcamiento fácil, plaza disponible en breve.
- **Amarillo** (10-30 min): aparcamiento moderado, puede requerir algo de búsqueda.
- **Rojo** (> 30 min): dificultad alta para aparcar, se recomienda considerar alternativas.

Este indicador es el más relevante para el usuario porque sintetiza en un solo color la información más crítica del trayecto: si va a poder aparcar o no cuando llegue.

4.2 Semáforo de tráfico

El semáforo de tráfico traduce la intensidad predicha por el modelo LSTM en un color, basándose en los niveles de servicio utilizados por el Ayuntamiento de Madrid:

- **Verde** (< 800 veh/hora): tráfico fluido.
- **Amarillo** (800-1.500 veh/hora): tráfico moderado.
- **Rojo** (> 1.500 veh/hora): congestión.

4.3 Indicador de confianza de la predicción

Informa al usuario sobre la fiabilidad de la predicción de NEXT PARK. La confianza es alta cuando el vehículo circula a menos de 40 km/h, el área cubierta es coherente con un comportamiento de búsqueda de aparcamiento y el conductor no está en una zona frecuente (donde el modelo no es fiable porque el conductor aparca habitualmente sin pasar por la fase de búsqueda). La confianza es baja cuando el conductor está en una zona frecuente o circula a más de 75 km/h.

4.4 Alerta de salida óptima

Compara el ETA actual con el ETA que obtendría el usuario si esperara 15, 30 o 45 minutos antes de salir. Si la diferencia de tráfico entre el momento actual y alguno de los intervalos futuros justifica la espera (es decir, si el ahorro en tiempo de conducción es mayor que el

tiempo de espera), el sistema informa al usuario: "Si esperas 15 min ahorras 8 min de trayecto". Si no merece la pena esperar, el sistema recomienda salir inmediatamente.

4.5 Indicador de ahorro de tiempo

Muestra al usuario cuánto tiempo ahorra usando el parking recomendado frente a la segunda mejor opción y frente a buscar aparcamiento en calle. El tiempo estimado de búsqueda en calle se fija en 15 minutos adicionales, valor conservador basado en estudios de movilidad urbana en ciudades españolas de tamaño similar.

4.6 Semáforo de zona de destino

Utiliza el dataset de velocidades máximas inferido sobre la red viaria de la Comunidad de Madrid (otro proyecto del PERTE VEC) para informar al usuario de la velocidad máxima en el área de destino. Esto permite al conductor anticipar si llegará a una zona 30, una zona urbana convencional o una vía rápida, lo que condiciona tanto la maniobra de aparcamiento como el tiempo de conducción en los últimos tramos.

4.7 Alerta de batería

Utiliza los datos de nivel de batería obtenidos del OBD del vehículo para estimar si la autonomía restante es suficiente para llegar al destino más el parking más cercano. El cálculo es sencillo: se estima la autonomía restante en kilómetros a partir del porcentaje de batería actual y la autonomía total del vehículo, y se compara con la distancia total necesaria (destino + parking). Si la batería es insuficiente, se genera una alerta inmediata. Si es suficiente pero con un margen inferior al 20%, se genera una alerta de precaución.

4.8 Indicador de punto de recarga cercano

Busca puntos de recarga eléctrica en un radio de 300 metros alrededor del parking recomendado. Si encuentra alguno, informa al usuario del nombre del punto, la distancia a pie, la potencia del cargador en kW, el tiempo de espera estimado basado en la ocupación del cargador y el tiempo aproximado para cargar un 30% de batería adicional a esa potencia. Este indicador es especialmente relevante para usuarios con batería baja que necesitan planificar una recarga durante su estancia en el destino.

5. Arquitectura del motor de integración

El motor de integración está implementado en Python y se estructura en cinco módulos independientes que se comunican a través de interfaces bien definidas:

TrafficPredictor es la interfaz con el modelo LSTM de Laura. Encapsula la carga del modelo, la preparación de features (incluyendo las transformaciones cíclicas) y la inferencia. En modo producción carga el modelo `.h5` entrenado; en modo desarrollo opera con una simulación paramétrica que reproduce los patrones de hora punta y hora valle.

NextParkPredictor implementa en Python el algoritmo NEXTPARK completo, con los coeficientes reales del modelo definitivo, la lógica de ventanas temporales para el cálculo de features, la detección de zonas frecuentes con caché en memoria para evitar lecturas repetidas de disco, y la evaluación de confianza de la predicción.

ETAEngine es el motor central de integración. Recibe las predicciones de los dos módulos anteriores, aplica el filtro de ocupación, calcula el ETA holístico para cada parking candidato y devuelve los resultados ordenados por tiempo total ascendente.

IndicadoresUX genera los semáforos básicos (aparcamiento, tráfico, confianza) y el resumen de texto optimizado para CarPlay.

IndicadoresUXAvanzados extiende el módulo anterior con los cinco indicadores adicionales: alerta de salida óptima, ahorro de tiempo, semáforo de zona, alerta de batería e indicador de recarga cercana.

6. Decisiones de diseño y justificación

A lo largo del desarrollo se tomaron varias decisiones de diseño que merecen justificación explícita.

Por qué usar distancia geodésica y no euclídea para la caminata. La distancia euclídea es una aproximación válida para distancias muy cortas en el mismo sistema de coordenadas planas, pero las coordenadas geográficas (latitud/longitud) no forman un sistema plano: un grado de longitud no mide lo mismo en kilómetros en Madrid que en el ecuador. La librería geopy resuelve esto calculando la distancia sobre el elipsoide terrestre, lo que garantiza precisión independientemente de la posición geográfica.

Por qué el umbral de ocupación es del 95% y no del 100%. Un parking al 100% de ocupación es formalmente inaccesible, pero en la práctica la ocupación reportada por los sensores puede tener un retardo de varios minutos respecto a la situación real. Un umbral del 95% introduce un margen de seguridad que evita enviar al usuario a un parking que estará lleno cuando llegue.

Por qué T_manobra es un parámetro fijo y no predicho. El tiempo de maniobra dentro de un parking (encontrar la rampa, circular por las plantas, aparcar) es difícil de predecir con precisión sin datos específicos de cada parking (número de plantas, distribución, ocupación por planta). El valor de 4 minutos es una estimación conservadora y robusta que puede ser personalizada por parking cuando se disponga de datos históricos de cada instalación.

Por qué se evalúan intervalos fijos (15, 30, 45 min) para la alerta de salida óptima. En un sistema de navegación orientado al conductor, la acción que se le propone debe ser concreta y accionable: "espera 15 minutos" es una instrucción clara; "espera el tiempo óptimo" no lo es. Los intervalos de 15, 30 y 45 minutos cubren el rango de esperas razonables en un contexto urbano sin sobrecargar al usuario con opciones.

7. Resultados de la simulación

Se ejecutó el motor con datos simulados en tres escenarios representativos:

Escenario 1 - Hora valle (10:15h): tráfico fluido (800 veh/h, 40 km/h), aparcamiento fácil (7 min, semáforo verde), batería al 75% con margen suficiente, zona 30 en el destino. ETA total: 12 minutos. El sistema recomienda salir inmediatamente y señala un punto de recarga a 140 metros del parking recomendado sin tiempo de espera.

Escenario 2 - Hora punta (08:30h): tráfico congestionado (2.200 veh/h, 15 km/h), aparcamiento moderado (17 min, semáforo amarillo), batería al 25% con margen justo. ETA total: 24 minutos. El sistema alerta sobre la batería baja y recomienda recargar en el punto disponible cerca del parking.

Escenario 3 - Zona saturada (19:00h): tráfico congestionado, aparcamiento difícil (35 min, semáforo rojo), batería al 60%. ETA total: 24 minutos de conducción más 35 minutos estimados de búsqueda de aparcamiento. El sistema recomienda considerar parkings alternativos o cambiar la hora de llegada.

En los tres escenarios el motor filtró correctamente el Parking Recoletos por ocupación crítica (98%) y recomendó el Parking Centro como opción óptima, con un ahorro de 6 minutos frente a la segunda opción y de aproximadamente 11 minutos frente a buscar aparcamiento en calle.

8. Próximos pasos y mejoras identificadas

Durante el desarrollo se identificaron varias líneas de mejora para iteraciones futuras:

- **Factor de corrección de caminata:** aplicar un factor de 1,25 sobre la distancia en línea recta para aproximar mejor la distancia real por las calles, o integrar una API de rutas peatonales (como OpenRouteService, que es de código abierto) para calcular la ruta real a pie.
- **T_maniobra dinámico:** parametrizar el tiempo de maniobra por parking a partir de datos históricos de cada instalación, permitiendo que el sistema aprenda que algunos parkings son más eficientes que otros.
- **Integración con el grafo de Dijkstra:** conectar el TrafficPredictor con el grafo de la red viaria para calcular el tiempo de conducción tramo a tramo en lugar de usar la velocidad media del nodo de destino.
- **Datos OBD en tiempo real:** integrar la lectura del nivel de batería directamente desde el OBD del vehículo para que la alerta de batería opere con datos reales en lugar de un parámetro de entrada manual.
- **Base de datos de puntos de recarga:** integrar el mapa de puntos de recarga de la Red de Puntos de Recarga de la Comunidad de Madrid o de plataformas abiertas como OpenChargeMap para que el indicador de recarga cercana opere con datos reales y actualizados.